Day 3: Tokens, Context Window aur Next-Token Prediction

Gen-AI + Agentic AI with JavaScript (Gemini Edition) | Phase 1: LLM Foundations

1. Concept

Token kya hota hai?

LLM **words** nahi padhta, wo **tokens** padhta hai. Token text ka ek chhota tukda hota hai (poora word, word ka hissa, ya ek symbol).

```
"unbelievable"  ->  ["un", "believ", "able"]   (3 tokens)
"Hello world!"  ->  ["Hello", " world", "!"]   (3 tokens)
```

Ye sirf example hai. Gemini ka asli tokenizer thoda alag tareeke se todta hai.

> **Rule of thumb:** Gemini models me ek token lagbhag 4 characters ke barabar hota hai, aur 100 tokens lagbhag 60-80 English words ke.

Hinglish note: Hinglish aur Hindi (Devanagari) text aam taur par English se **zyada tokens** leta hai, kyunki tokenizer English pe zyada optimize hota hai. Code me khud measure karo.

Tokens itne important kyun hain?

Wajah	Matlab
Cost	Billing on hone par API call ka cost input aur output tokens pe depend karta hai. Free tier me paisa nahi lagta, lekin **quota tokens ke hisaab se bhi khatam hota hai.**
Limit	Har model ka maximum token limit hota hai (context window).
Speed	Zyada tokens = zyada time.

Next-Token Prediction: LLM ka asli kaam

LLM ka sirf ek kaam hai: **"Ab tak ke text ke baad agla token kya hona chahiye?"** Analogy: phone keyboard ka autocomplete, bas bahut bada aur smart.

```
"Aaj mausam bahut"  ->  model har possible next token ko probability deta hai:
                        "accha" (45%), "kharab" (20%), "garam" (15%), ...
                    ->  ek token chunta hai  ->  "accha"
"Aaj mausam bahut accha"  ->  phir agla token...
```

Ye loop tab tak chalta hai jab tak model "stop" na kare ya maxOutputTokens ki limit na aa jaye. Isi wajah se response **stream** ho sakta hai (Day 6), aur isi wajah se same prompt par alag jawab aa sakte hain, kyunki token chunne me thodi randomness hoti hai (temperature, Day 4).

Context Window: model ki short-term memory

Context window = ek call me model jitna total text (input + output) dekh sakta hai. Sabse important baat: **LLM stateless hota hai.** Wo pichli baat yaad nahi rakhta. Chatbot me har naye message ke saath **poori purani history dobara bheji jaati hai.**

```
Turn 1: [Q1]                      -> 50 tokens
Turn 2: [Q1 + A1 + Q2]            -> 300 tokens
Turn 3: [Q1 + A1 + Q2 + A2 + Q3]  -> 700 tokens   <- history badhti jaati hai!
```

> Lambi chat = zyada tokens = zyada quota/cost, aur ek point par context window bhar jayegi. Isliye Day 23 me chat memory aur summarization seekhenge.

Thinking tokens (Gemini ka extra)

Thinking model me jo tokens "sochne" me lagte hain, wo alag `thoughtsTokenCount` me aate hain. Ye response text me nahi dikhte, lekin quota me count hote hain. Isliye ek chhota sawal bhi kabhi-kabhi expected se zyada tokens kha leta hai.

2. Workflow

```
Tumhara text
   |
   v
Tokenizer  ->  tokens (numbers/IDs)
   |
   v
LLM  ->  agle token ki probabilities
   |
   v
Ek token chuna -> output me jodo
   |
   +---- repeat ----+
                    v
   Stop token mila ya maxOutputTokens hua? -> Response done
```

Developer ke liye 2 tools

1. `countTokens` ko request bhejne se pehle call karke input ka size check kar sakte ho.
2. `generateContent` ke baad `usageMetadata` se input aur output dono ka token count milta hai.

usageMetadata ke fields

Field	Matlab
`promptTokenCount`	Input tokens
`candidatesTokenCount`	Output tokens
`thoughtsTokenCount`	Thinking tokens (agar model sochta hai)
`totalTokenCount`	Sab milake

3. Code

Folder: phase-1-foundations/day-03. Setup Day 2 jaisa hi hai (npm init -y, "type": "module", npm i @google/genai dotenv). .env aur config.js Day 2 se copy kar lo.

count-tokens.js: English vs Hinglish vs Hindi

```
import "dotenv/config";
import { GoogleGenAI } from "@google/genai";
import { MODEL } from "./config.js";

const ai = new GoogleGenAI({});

const samples = {
  English: "The weather is very nice today, let's go for a walk.",
  Hinglish: "Aaj mausam bahut accha hai, chalo walk pe chalte hain.",
  Hindi: "<Devanagari me ek Hindi sentence yahan likho>",
  Code: "const sum = (a, b) => a + b;",
  JSON: '{"name":"Rahul","age":25,"skills":["React","Node"]}',
};

for (const [label, text] of Object.entries(samples)) {
  const res = await ai.models.countTokens({ model: MODEL, contents: text });
  const ratio = (text.length / res.totalTokens).toFixed(1);
  console.log(
    `${label.padEnd(9)} | chars: ${String(text.length).padStart(3)} | tokens: ${String(res.totalTokens).pa
```

```
  );
}
```

Dekhna: Kaunsi language me chars/token sabse kam aata hai? Matlab wahi sabse "mehngi" hai. (Hindi line me Devanagari sentence khud likhna, jaise "aaj mausam bahut accha hai" Hindi script me.)

usage.js: ek call ka poora hisaab

```
import "dotenv/config";
import { GoogleGenAI } from "@google/genai";
import { MODEL } from "./config.js";

const ai = new GoogleGenAI({});

const response = await ai.models.generateContent({
  model: MODEL,
  contents: "Explain tokens in one short paragraph.",
});

const u = response.usageMetadata;
console.log("Answer:", response.text);
console.log("--- Usage ---");
console.log("Input tokens   :", u?.promptTokenCount);
console.log("Output tokens  :", u?.candidatesTokenCount);
console.log("Thinking tokens:", u?.thoughtsTokenCount ?? 0);
console.log("Total          :", u?.totalTokenCount);
```

Dekhna: Kya Input + Output = Total hota hai? Agar nahi, toh thinking tokens ka role samjho.

context-growth.js: history kaise badhti hai

```
import "dotenv/config";
import { GoogleGenAI } from "@google/genai";
import { MODEL } from "./config.js";

const ai = new GoogleGenAI({});

// chat object history khud sambhalta hai aur har turn pe poori history bhejta hai
const chat = ai.chats.create({ model: MODEL });

const questions = [
  "What is Node.js? Answer in 3 lines.",
  "And what is Express? Answer in 3 lines.",
  "Compare both in 3 lines.",
];

for (const q of questions) {
  const res = await chat.sendMessage({ message: q });
  console.log(`Q: ${q}`);
  console.log(`   Input tokens this turn: ${res.usageMetadata?.promptTokenCount}`);
}
```

Dekhna: Har turn me Input tokens badh rahe hain, jabki naya sawal chhota hi hai. Yahi "history dobara bheji jaati hai" ka proof hai.

model-limits.js: apne model ki context window

```
import "dotenv/config";
import { GoogleGenAI } from "@google/genai";
import { MODEL } from "./config.js";

const ai = new GoogleGenAI({});

try {
  const info = await ai.models.get({ model: MODEL });
  console.log("Model       :", info.name);
```

```
  console.log("Input limit :", info.inputTokenLimit);
  console.log("Output limit:", info.outputTokenLimit);
} catch (err) {
  console.error("Model info nahi mili:", err.message);
}
```

Note: countTokens bhi ek API call hai aur uski bhi limits hoti hain. Isliye loops me bahut saari calls mat chalana.

Homework

1. Chaaron scripts **khud type karke** chalao.
2. count-tokens.js me apna ek real prompt aur ek lamba paragraph daalo, aur dekho kitne tokens lagte hain.
3. context-growth.js me 5 turns tak badhao aur growth ka pattern note karo.
4. Ek hi sawal "Hinglish me" aur "in English" puchho, aur usageMetadata compare karo.

4. Quick Revision (Quiz Answers)

Question	Answer
Token ke baare me sahi statement?	Token text ka chhota hissa hota hai, lagbhag 4 characters
LLM ka core kaam?	Ab tak ke text ke baad agla token predict karna
Chat me input tokens kyun badhte hain?	Har turn pe poori purani history dobara bheji jaati hai
Token count kahan se milta hai?	response.usageMetadata
Input 11 + Output 47, Total 881. Wajah?	Thinking tokens (thoughtsTokenCount) bhi total me jud gaye

Next: Day 4: Parameters: temperature, topP, topK, maxOutputTokens